Character-Based Sequence-to-Sequence Modelling for Ukrainian Grammatical Error Correction

Viktoriia Tsosenko
Heinrich Heine University Düsseldorf
Viktoriia.Tsosenko@hhu.de

Abstract

Grammatical error correction (GEC) has been studied successfully for English but remains a challenging task for morphologically complex languages such as Ukrainian. In this work, we present a character-level sequence-to-sequence model based on a bidirectional LSTM encoder and an LSTM decoder for Ukrainian GEC. We train and evaluate our model on the manually annotated UA-GEC dataset, consisting of 33,735 sentences and covering grammatical, spelling, punctuation, and fluency errors. We then use a fine-tuned mBART50 baseline and compare it against our model.

1 Introduction

Grammatical Error Correction (GEC) in Natural Language Processing (NLP) performs automatic error detection and correction in written texts. The main error types are (Bryant et al., 2017):

(a) orphographic (spelling and punctuation),

(b) morphological (word ending or forms),

(c) syntactic (word order or missing words),

(d) lexical errors (wrong context or phrasing).

Interest in Ukrainian NLP has grown significantly in recent years. Despite this, Ukrainian remains a low-resource language due to the lack of annotated corpora (Pogorilyy and Kramov, 2020). In this work we use the first and currently the only dataset for Ukrainian, UA-GEC. Because of its morphological complexity (e.g. deep system of cases, verbal aspects), Ukrainian GEC is a challenging task for token-based models (Syvokon et al., 2023). Unlike English, Ukrainian is an inflectional language. The most common errors in Ukrainian happen on the morphological level (wrong word endings and agreement) (Zuban, 2017) and lexical level (calques). Plenty of Ukrainians are native speakers in both Ukrainian and Russian, which arises from a complex historical and political background. A widespread phenomenon in Ukraine is code-switching between the two languages, referred to as *syrzhyk*. It occurs across multiple linguistic levels blending vocabulary, grammar, and pronunciation from both Ukrainian and Russian (Hentschel and Reuther, 2020). Due to such complexity, we use a character-based approach in our paper. We also compare the model against a fine-tuned mBART50 baseline (Tang et al., 2021), since this model achieved the best results in the UA-GEC shared task (Syvokon and Romanyshyn, 2023a).

To summarize, our contributions are as follows:

- Train a character-based encoder-decoder model on UA-GEC and evaluate it with the F0.5 metric.
- Compare our model against the fine-tuned mBART50.
- Analyze performance based on error types and discuss the pros and cons of a character-based approach for a fusional language.

2 Dataset

Ukrainian belongs to low-resource languages and UA-GEC is the first professionally annotated dataset for Ukrainian GEC. It consists of written texts, both from native and non-native speakers, and annotated by two different annotators. The annotation has the following format: `{error=>edit:::error_type=Tag}`, where *error* is the original text, *edit* is the correction, and *Tag* denotes the error category (Syvokon et al., 2023).

Errors in UA-GEC are divided into four types, namely grammar, fluency, spelling and punctuation. There are two corpus versions in `m2` format, namely `gec only` and `gec + fluency`. In this paper, we concentrate on the latter one. `gec + fluency` is di-

vided into train and test sets, with 31,038 and 2,697 sentences, respectively (Syvokon et al., 2023).

The `m2` format is standard in GEC and its structure is shown in Table 1. Line S contains a sentence and A-lines are annotations stating error location, it's type and the corresponding correction, made by two different annotators.

Line	Content
S	She go to the store every day .
A	1 2 ||| G/Number ||| goes ||| REQUIRED ||| -NONE- ||| 0
A	1 2 ||| G/Number ||| goes ||| REQUIRED ||| -NONE- ||| 1

Table 1: An example of a m2-formatted sentence.

To use the sentences in training, they must be parsed first. UA-GEC repo provides already parsed sentences as source and target text files, so no additional parsing is required (Syvokon and Romanyshyn, 2023a).

3 Models

3.1 Character-Level Encoder-Decoder

As the model's name suggests, it is a character-level model consisting of two parts, namely the Encoder and Decoder.

3.1.1 Character-Level Vocabulary

There are several approaches to text segmentation for seq2seq models: word-level, subword level (e.g. BPE or WordPiece), and character-level tokenization. A word-level model treats different inflected forms of the same word (e.g. different cases or moods) as completely different tokens, which is problematic for fusional languages as Ukrainian (Syvokon et al., 2023). BPE merges character sequences from the training data based on how often they appear together into single tokens. As a result, a model may end up cutting the Ukrainian word in the wrong place and thus deviate a morpheme boundaries (Asgari et al., 2025). Character-level models are more reliable at capturing the structure of morphologically rich languages than BPE, and can outperform subword segmentation entirely (Shapiro and Duh, 2018). They allow the model to recognize grammatical function of inflections and are better with orthographic errors (Xie et al., 2016). Given these considerations, we adopt a character-level approach for a fusional language like Ukrainian.

To build the vocabulary, we used the train dataset of both source and target sentences. As a result, we received 322 distinct characters, of which only 70 are Cyrillic. Although the Ukrainian alphabet consists of 33 letters, the model treats their upper and lower case forms as separate symbols (66 in total), plus there are four Russian-only letters found in the data (Э, э, ы, ё). The remaining characters that explain such a high count are: 61 Latin letters, 16 digits (including superscript) and 175 other symbols such as emojis, punctuation and other signs.

Additionally, there are four special symbols we added at the beginning of the vocabulary so that the model can indicate padding (`<PAD>`), start of the sentence (`<SOS>`), end of the sentence(`<EOS>`), and any unknown symbols (`<UNK>`). The (`<UNK>`) symbol is reserved for any characters unseen during training but present in future validation and inference steps. Thus the final vocabulary for Ukrainian GEC contains 326 symbols in total.

The model cannot work with those symbols directly, it requires a numerical input. Thus, every symbol receives its own numerical ID (see Table 2). Each ID is then mapped to a learned embedding vector required by the encoder and decoder.

Token	Index
<PAD>	0
<SOS>	1
<EOS>	2
<UNK>	3
'а'	4
'б'	5
'в'	6
⋮	⋮

Table 2: Token-to-index mapping. Indices 0–3 are reserved for special tokens; character indices begin at 4.

3.1.2 Encoder

The encoder reads erroneous Ukrainian sentences, one character at a time and converts each character into a vector. Each character ID first attaches to a learned embedding vector through an embedding layer. Next, the embeddings are processed by *a single-layer bidirectional LSTM*, meaning it reads each sentence in both directions, left-to-right and right-to-left. This lets the model understand each character better, based not only on the context before the character, but also on what follows. For Ukrainian it plays an important role, since agreement and case endings often rely on the words that come after. There are two outputs that the encoder produces: a reading of every character (used later

by attention) and the final hidden and cell states of a fixed length (summary).

Forward and backward readings form two hidden states that are concatenated into one. Similarly within the cell states, their directions get concatenated and become one cell state. After concatenation the size of both states needs to be reduced to the decoder's expected size. By applying two separate *linear projections*, followed by a *tanh activation*, we receive the decoder's starting point.

3.1.3 Attention Mechanism

Without attention, the encoder must compress everything it read into a single summary vector that is handed to the decoder to generate its output. Since this vector has a fixed size, longer sentences lose some information in order to fit in. Such a squeezed vector can result in the decoder's degraded accuracy (Bahdanau et al., 2015). To solve this problem we use Bahdanau's attention mechanism that lets the decoder look back at every encoder output directly, and weight source characters to how relevant they are at that moment.

3.1.4 Decoder

The decoder's task is to write correct sentences. When it receives the encoder's summary, it starts predicting correct characters, one at a time, left-to-right. By reading the special tokens `<SOS>` and `<EOS>` the decoder knows when the sentence begins and ends. Our decoder contains an attention mechanism and thus it is no longer limited to the encoder's summary only. With attention, it builds the context vector at every single step of the decoding process. Both the context vector and the previous character's embedding are fed into the decoder's LSTM as well as the final output projection. This allows the model to look back and notice what source characters are the most relevant at the current step. The output is generated sequentially and not all at once because the context relies on the decoder's constantly updating hidden state (Bahdanau et al., 2015).

During training the decoder makes predictions (called *logits*), but they are never fed back into the model. Instead, *teacher forcing* is used, meaning the model always receives the correct previous characters. This enables a faster training process.

The inference time is where the decoder uses its own predictions and actually demonstrates what is has learned. After training, the best model is saved and loaded again to correct new sentences at inference time.

3.1.5 Sequence-to-Sequence

Sequence-to-Sequence (Seq2Seq) is what actually brings the encoder and the decoder with attention mechanism to life. It is a wrapper that connects the two of them into one model and defines how the training and inference proceed. It also masks `<PAD>` tokens so that the attention mechanism ignores them.

3.2 mBART50

mBART50 is a transformer based encoder-decoder pretrained on 50 languages, including Ukrainian. We use `mbart-large-50` that has about 610M parameters (Tang et al., 2021). It does not rely on a character vocabulary as the character-level LSTM does, since it uses its own pretrained SentencePiece subword tokenizer (Liu et al., 2020). The model has already seen plenty of Ukrainian words in pretraining, and thus, it contains representations of most Ukrainian word forms, unlike with the LSTM that starts training with zero language knowledge. By fine-tuning mBART50, the existing representations are adapted to the correction task. It is important to note, that the subword tokenization may still cut morpheme boudaries since it it only frequency-based and has no knowledge of what a morpheme actually is.

4 Training LSTM vs Fine-tuning mBART50

The hardware used for both training and finetuning was 16 GB RTX 4060 Ti, using the `gec+fluency` training split. The hyperparamenetrs are summarized in Table 3.

4.1 Training the LSTM

LSTM trained 21 epochs (early-stopped, best at epoch 16, valid loss 0.1643, char accuracy 0.965)

4.2 Fine-tuning mBART50

mBART was fine-tuned only 3 epochs (best eval loss 0.2024 at epoch 2).

For tokenization and evaluation we used the corresponding publicly available scripts from the UA-GEC git (Syvokon and Romanyshyn, 2023a).

5 Results

The vocabularies of both models differ drastically: 326 tokens VS 250,000 tokens in favor of mBART50).

Setting	LSTM	mBART50
Parameters	∼4M (est.)	∼610M
Tokenization	character	SentencePiece
Optimizer	Adam	AdamW
Learning rate	1×10^{-3}	3×10^{-5}
Effective batch	48	32
Max length	800 chars	256 tokens
Precision	fp32	bf16
Best epoch	16	2
Decoding	greedy	beam (5)

Table 3: Training and fine-tuning configuration for the character-level LSTM and the fine-tuned mBART50 baseline.

Model	Precision	Recall	F0.5
Span-based correction			
LSTM (char-level)	0.4805	0.2600	0.4108
mBART50 (fine-tuned)	0.6997	0.3744	**0.5961**
Span-based detection			
LSTM (char-level)	0.5790	0.3033	0.4899
mBART50 (fine-tuned)	0.7891	0.4110	**0.6665**

Table 4: ERRANT evaluation on the UA-GEC validation set, comparing the character-level LSTM against a fine-tuned mBART50.

References

Ehsaneddin Asgari, Yassine El Kheir, and Mohammad Ali Sadraei Javaheri. 2025. MorphBPE: A morpho-aware tokenizer bridging linguistic complexity for efficient LLM training across morphologies. *arXiv preprint arXiv:2502.00894*.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.

Maksym Bondarenko, Artem Yushko, Andrii Shportko, and Andrii Fedorych. 2023. Comparative study of models trained on synthetic data for Ukrainian grammatical error correction. In *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, pages 103–113, Dubrovnik, Croatia. Association for Computational Linguistics.

Christopher Bryant, Mariano Felice, and Ted Briscoe. 2017. Automatic annotation and evaluation of error types for grammatical error correction. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 793–805, Vancouver, Canada. Association for Computational Linguistics.

Gerd Hentschel and Tilmann Reuther. 2020. Ukrainisch-russisches und russisch-ukrainisches code-mixing. untersuchungen in drei regionen im süden der ukraine: Ein dreijähriges forschungsprojekt im rahmen des d-a-ch-programms von fwf und dfg. *Colloquium: New Philologies*, 5(2).

Carl James. 1998. *Errors in Language Learning and Use: Exploring Error Analysis*, 1 edition. Routledge, London.

Satoru Katsumata and Mamoru Komachi. 2020. Stronger baselines for grammatical error correction using a pretrained encoder-decoder model. In *Proceedings of the 1st Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 10th International Joint Conference on Natural Language Processing*, pages 827–832, Suzhou, China. Association for Computational Linguistics.

Lukasz Kulowski. 2019. LSTM encoder-decoder time series forecasting. `https://github.com/lkulowski/LSTM_encoder_decoder`. Accessed: 2026-05-08.

Yinhan Liu, Jiatao Gu, Naman Goyal, Xian Li, Vitaliy Murdock, Marjan Ghazvininejad, Luke Zhou, Edunov Sergey, and Luke Zettlemoyer. 2020. Multilingual denoising pre-training for neural machine translation. *Transactions of the Association for Computational Linguistics*, 8:726–742.

S. D. Pogorilyy and A. A. Kramov. 2020. Method of noun phrase detection in Ukrainian texts. *Control Systems and Computers*, (5):48–59.

Pamela Shapiro and Kevin Duh. 2018. Bpe and charcnns for translation of morphology: A cross-lingual comparison and analysis. *arXiv preprint arXiv:1809.01301*.

Oleksiy Syvokon, Olena Nahorna, Pavlo Kuchmiichuk, and Nastasiia Osidach. 2023. UA-GEC: Grammatical error correction and fluency corpus for the Ukrainian language. In *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, pages 96–102, Dubrovnik, Croatia. Association for Computational Linguistics.

Oleksiy Syvokon and Mariana Romanyshyn. 2023a. UA-GEC: Grammatical Error Correction Corpus for Ukrainian. `https://github.com/grammarly/ua-gec`. Accessed: 2026-04-27.

Oleksiy Syvokon and Mariana Romanyshyn. 2023b. The UNLP 2023 shared task on grammatical error correction for Ukrainian. In *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, pages 132–137, Dubrovnik, Croatia. Association for Computational Linguistics.

Yuqing Tang, Chau Tran, Xian Li, Peng-Jen Chen, Naman Goyal, Vishrav Chaudhary, Jiatao Gu, and Angela Fan. 2021. Multilingual translation from denoising pre-training. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3450–3466, Online. Association for Computational Linguistics.

Ziang Xie, Anand Avati, Naveen Arivazhagan, Dan Jurafsky, and Andrew Y. Ng. 2016. Neural language correction with character-based attention. *arXiv preprint arXiv:1603.09727*.

Oksana Zuban. 2017. Automatic morphemic analysis in the corpus of the Ukrainian language: Results and prospects. *Jazykovedný časopis [Journal of Linguistics]*, 68(2):415–426.